Objects and Classes

OO Programming in Java

Other than primitive data types (*byte, short, int, long, float, double, char, boolean*), everything else in Java is of type object.

Objects we already worked with:

String:
```
String name = new String("John Smith");
```

Scanner:
```
Scanner input = new Scanner(System.in);
```

Random:
```
Random generator = new Random(100);
```

Date:
```
Date date = new Date();
System.out.println(date.toString());
```

What is an Object?

An *object* represents an entity in the real world that can be distinctly identified. For example, student, desk, circle, button, person, course, car, employee, department, store, computer, etc…

For instance, an object might represent a particular employee in a company. Each employee object handles the processing and management of data related to that employee.

An object has a unique identity, state, and behaviors.

The *state* of an object consists of a set of *data fields* (*instance variables* or *properties*) with their current values.

The *behavior* of an object is defined by a set of methods defined in the class from which the object is created.

A ***class*** describes a set of similar objects.

What is a Class?

A class is the blueprint (template) that defines objects of the same type, a set of similar object, such as students.

The class uses methods to define the behaviors of its objects.

The class that contains the main method of a Java program represents the entire program

A class provides a special type of methods, known as constructors, which are invoked to construct (create) objects from the class.

Multiple objects can be created from the same class.

Constructor Methods

The contractor method creates the object in the memory with the help of the Operating System.

Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.

A class can have multiple versions of the constructor method, allowing the user to create the object in different ways.

The constructor method must have same name as the class name.

Constructors do not have a return type, not even **`void`**.

A constructor with no parameters is called *no-arguments constructor*.

Default Constructor

A class may be declared without constructors.

This constructor, called *a default constructor*, is provided automatically *only if no constructors are explicitly declared in the class*.

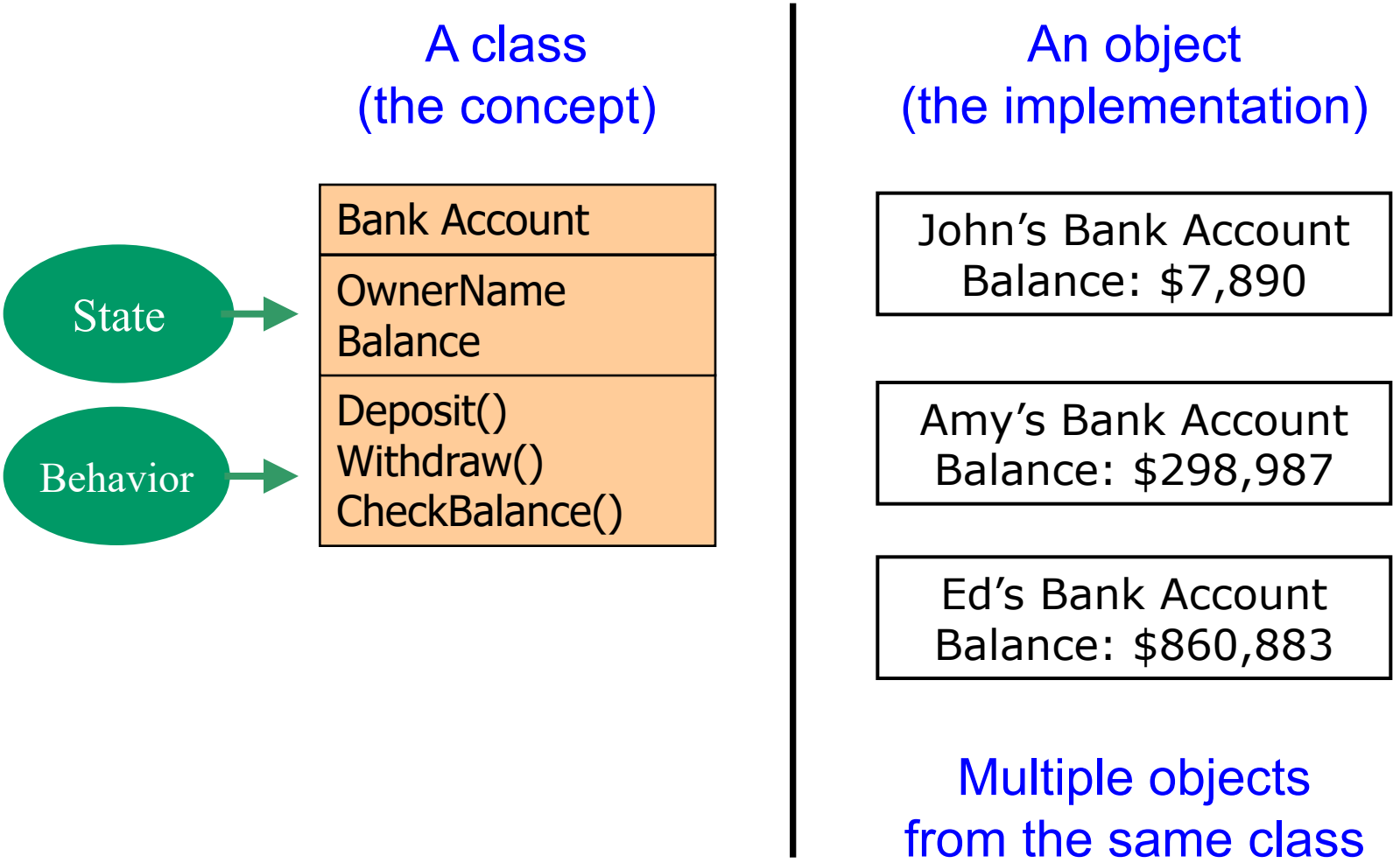
Writing Classes

The programs we've written in previous examples have used classes defined in the Java standard class library.

Now, we will begin to design programs that rely on classes that we write ourselves.

The class that contains the **`main`** method is just the starting point of a program. This is also known as the driver/test program

Example


A class
(the concept)
An object
(the implementation)
State
Behavior
Bank Account
OwnerName
Balance
Deposit()
Withdraw()
CheckBalance()
John's Bank Account
Balance: $7,890
Amy's Bank Account
Balance: $298,987
Ed's Bank Account
Balance: $860,883
Multiple objects
from the same class

Class Circle Constructors

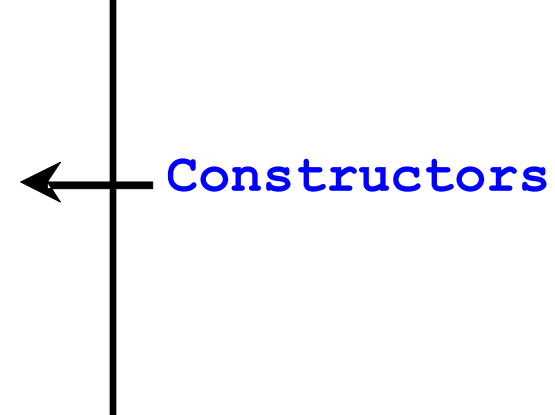
```
class Circle {
  // The radius of this circle
  double radius = 1.0;

  // Construct a circle object
  Circle() {
  }

  // Construct a circle object
  Circle(double newRadius) {
    radius = newRadius;
  }

  // Return the area of this circle
  double getArea() {
    return radius * radius * 3.14159;
  // other methods
  }
}
```







Creating Objects

To reference an object, assign the object to a reference variable.

To declare a reference variable, use the syntax:

```
ClassName objectRefVar;
```

Example:

```
Circle circle1, circle2;     //reference variables
circle1 = new Circle();      //calls first constructor
circle2 = new Circle(25.0); //calls second constructor
```

OR

```
Circle circle1 = new Circle();
Circle circle2 = new Circle(25.0);
```

Accessing the Object

Referencing the object's data fields:

```
objectRefVar.data
```

```
double radius1 = circle1.radius; //data field
```

Invoking the object's method:

```
objectRefVar.methodName(arguments)
```

```
double area1 = mcircle1.getArea(); //class method
```

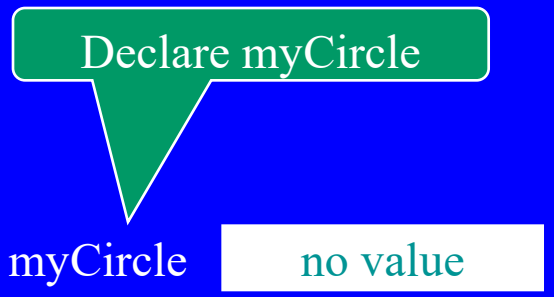
animation

Trace Code

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



animation

Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```

myCircle | no value

: Circle

radius: 5.0

Create a circle

animation

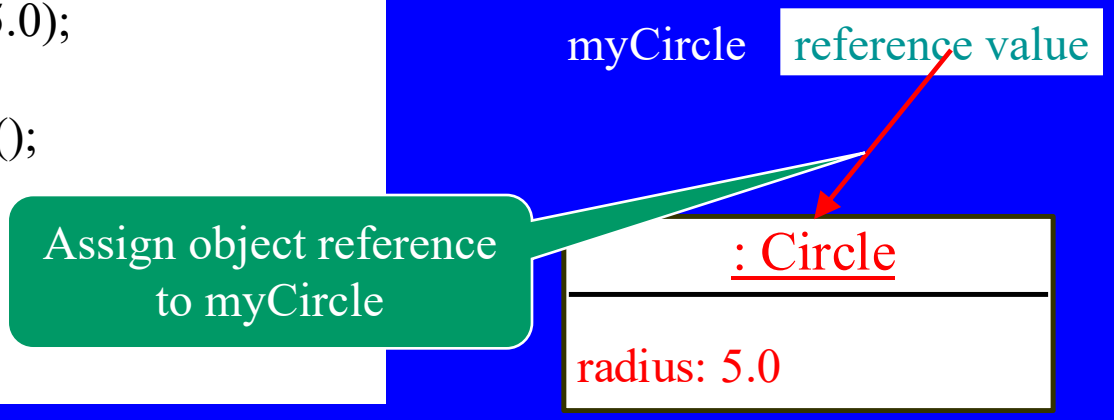
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```



animation

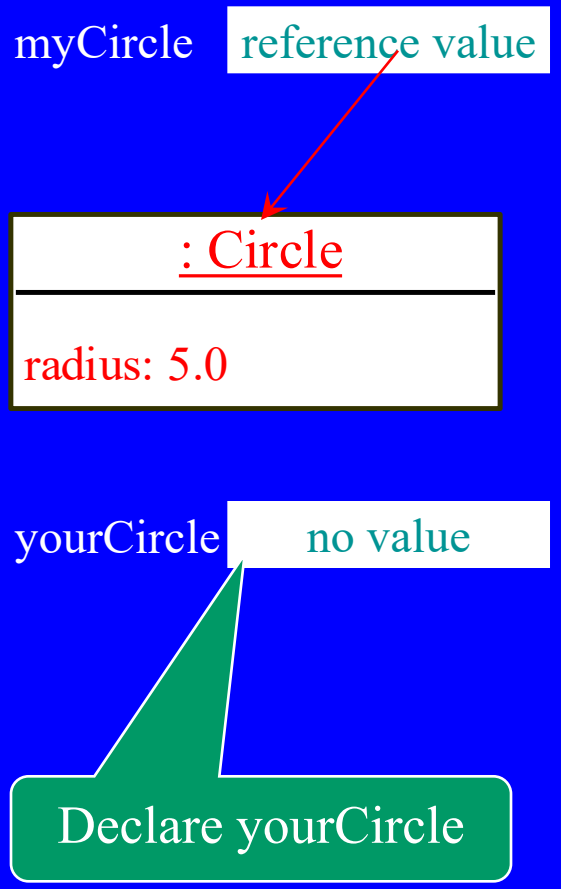
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```



animation

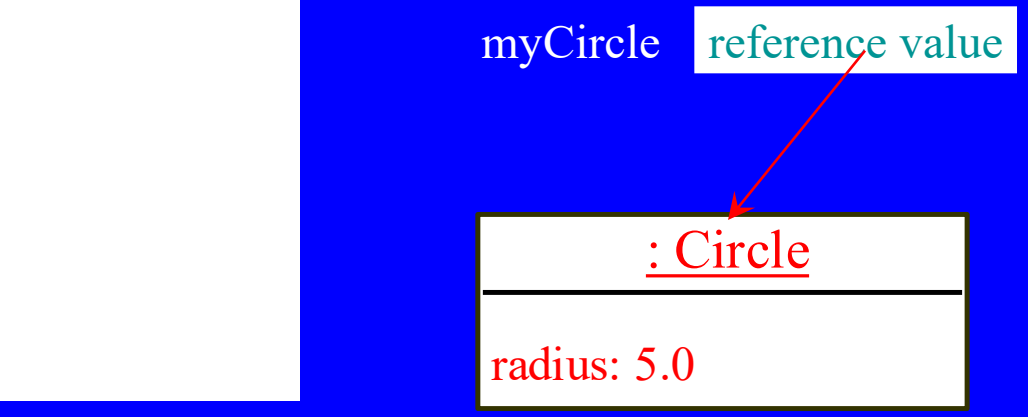
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

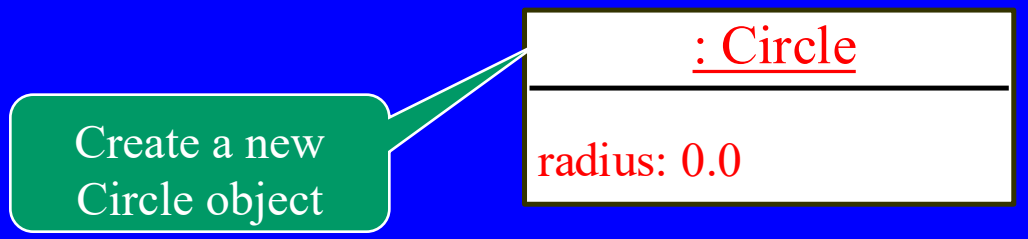
Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```



yourCircle — no value



animation

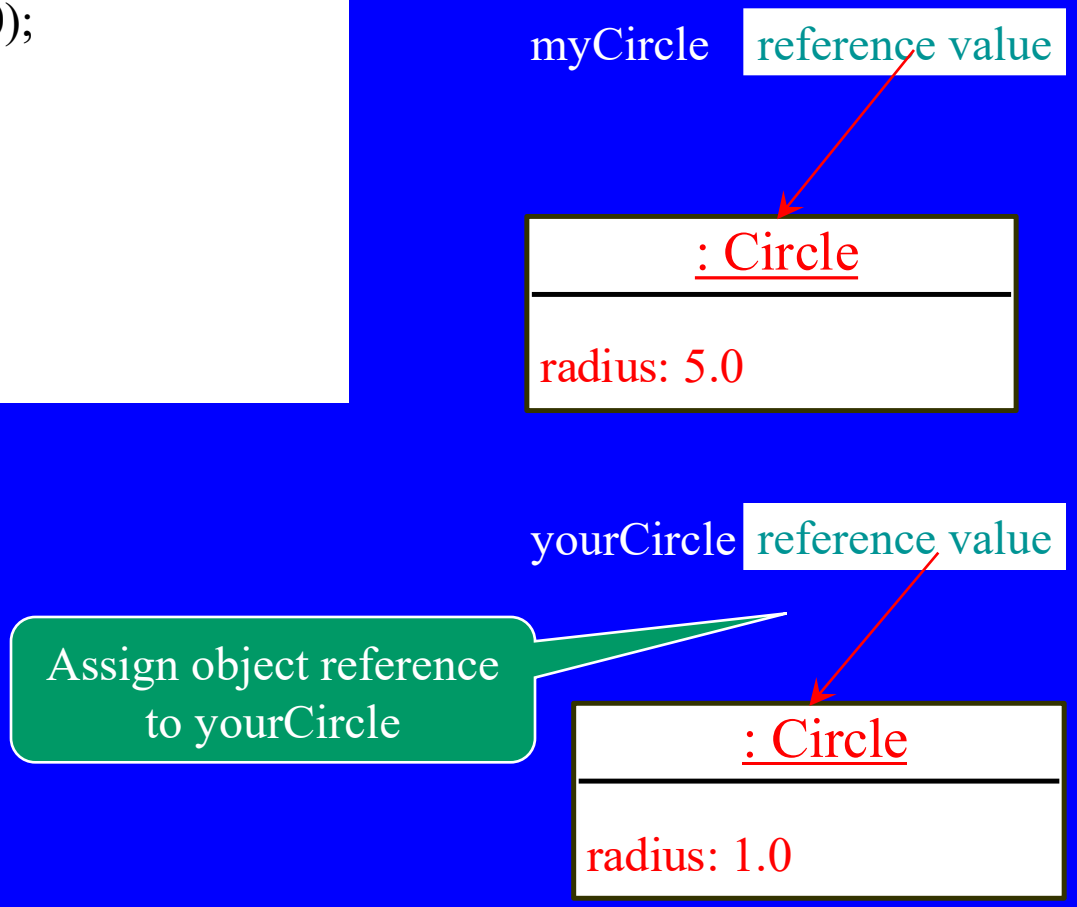
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```



animation

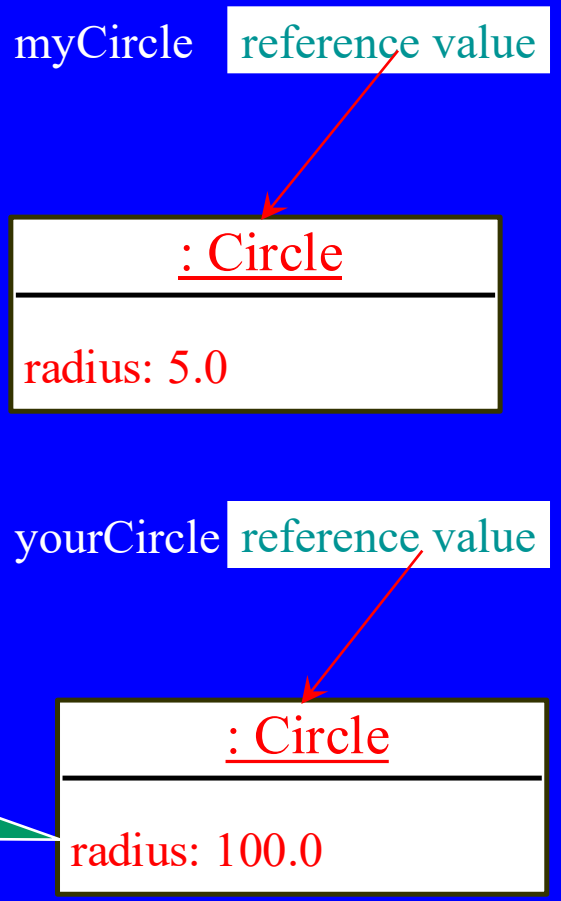
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```



animation

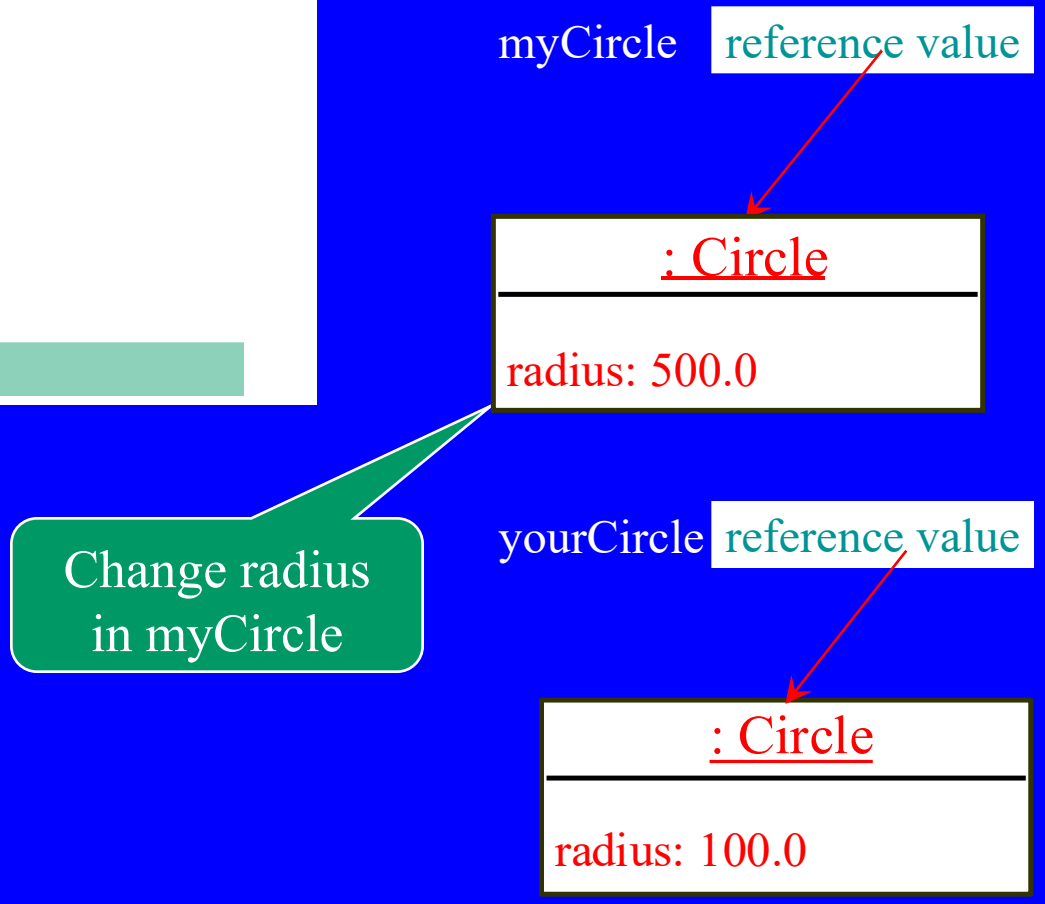
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle.radius = 500.0;
```



Caution

Recall that we used

```
Math.methodName(arguments)
(e.g., Math.pow(3, 2.5))
```

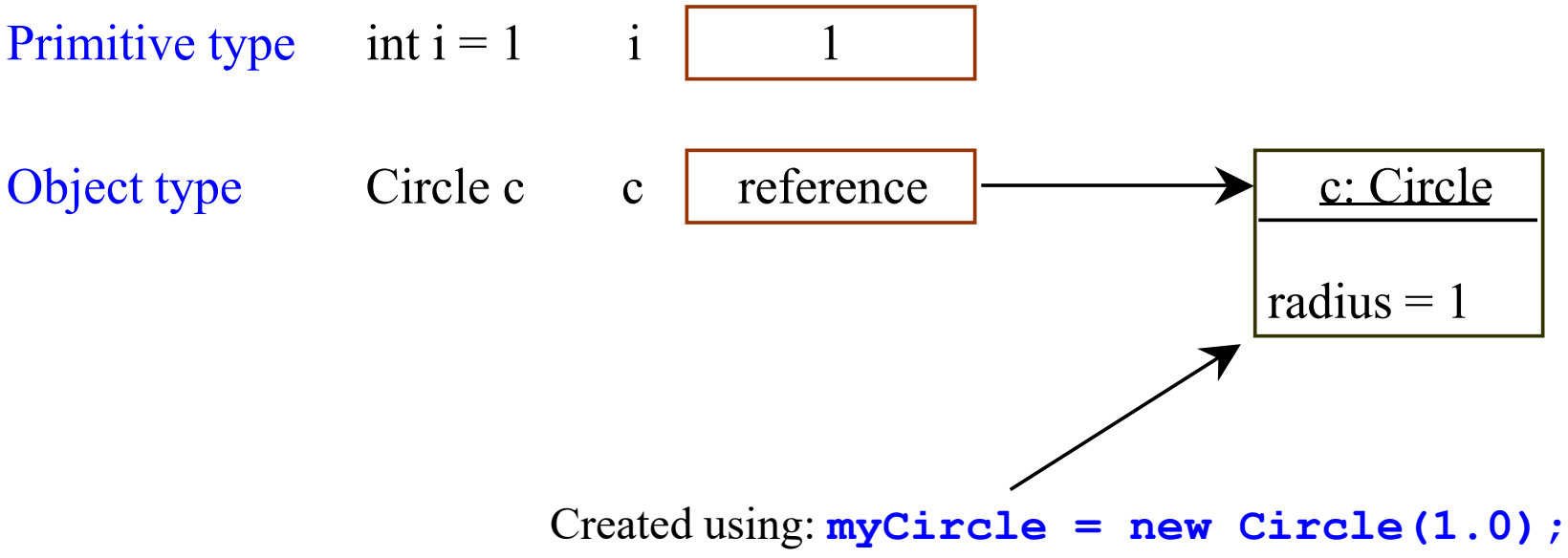
to invoke a method in the Math class.

Can you invoke `getArea()` using `Circle1.getArea()`?

All the methods defined in the Math class are static (defined using the **`static`** keyword). However, method `getArea()` is non-static. It must be invoked from an object using this syntax:

```
objectRefVar.methodName(arguments)
(e.g., myCircle.getArea())
```

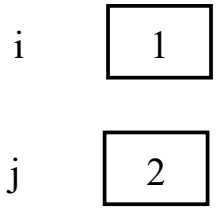
Primitive Type vs. Object Type



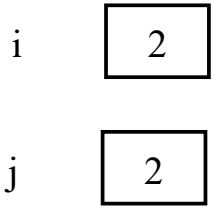
Primitive Type vs. Object Type

Primitive type assignment i = j

Before:

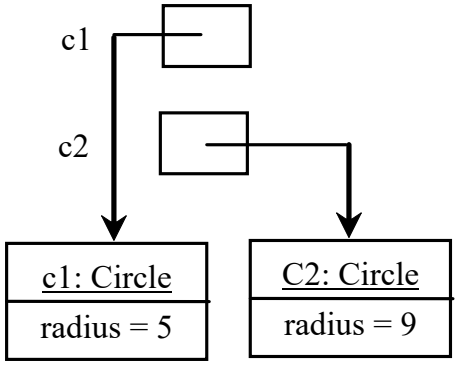


After:

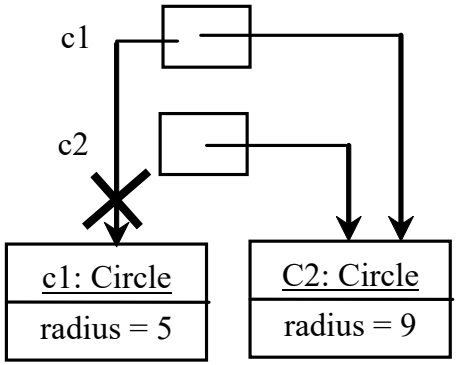


Object type assignment c1 = c2

Before:



After:



Static Variables, Constants, and Methods

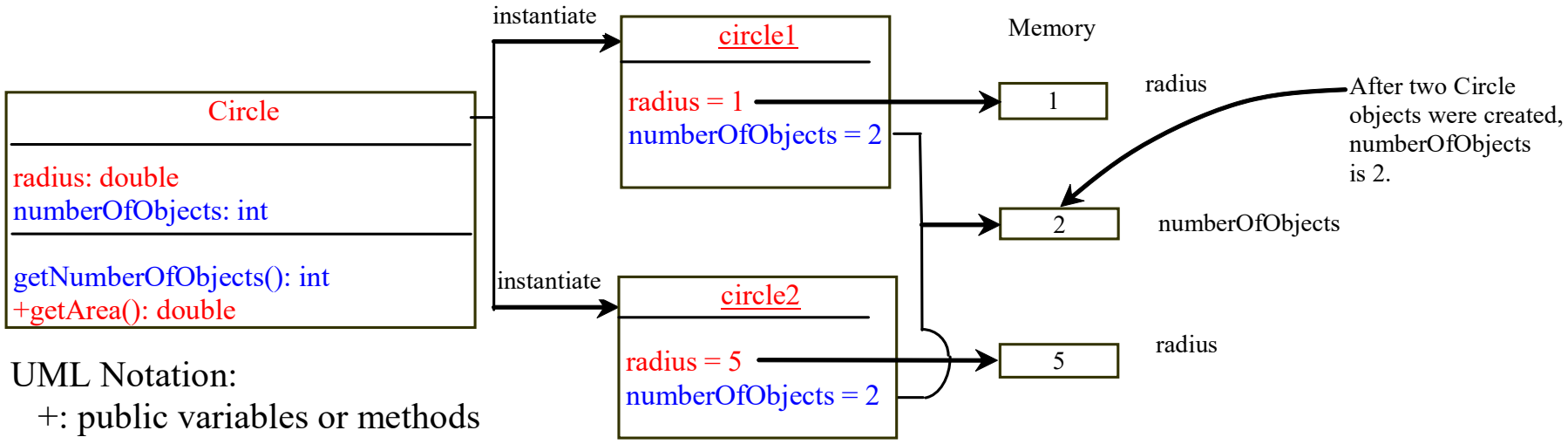
Static variables are shared by all the objects of the class.

Static methods are not tied to a specific object, applied to all objects of the class.

Static constants (`final` variables) are shared by all the objects of the class.

To declare static variables, constants, and methods, use the **`static`** modifier.

Static Variables, Constants, and Methods



UML Notation:

+: public variables or methods

blue: static variables or methods

Visibility Modifiers

By default (no qualifier), the class variable or method can be accessed by any class in the same package, but not other packages.

`Public:`

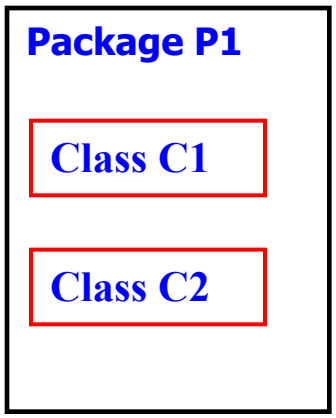
The class, data, or method is visible to any class in any package.

`Private:`

The data or methods can be accessed only by the declaring class.

The **`get`** and **`set`** methods are used to read and modify private variables.

Visibility Modifiers – Example 1




```
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```
package p1;

public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

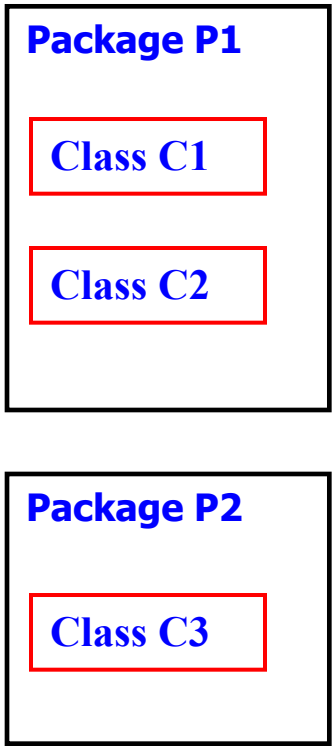
```
package p1;

class C1 {
  ...
}
```

```
package p1;

public class C2 {
  can access C1
}
```

Visibility Modifiers – Example 1




```
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```
package p2;

public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```
package p1;

class C1 {
  ...
}
```

```
package p2;

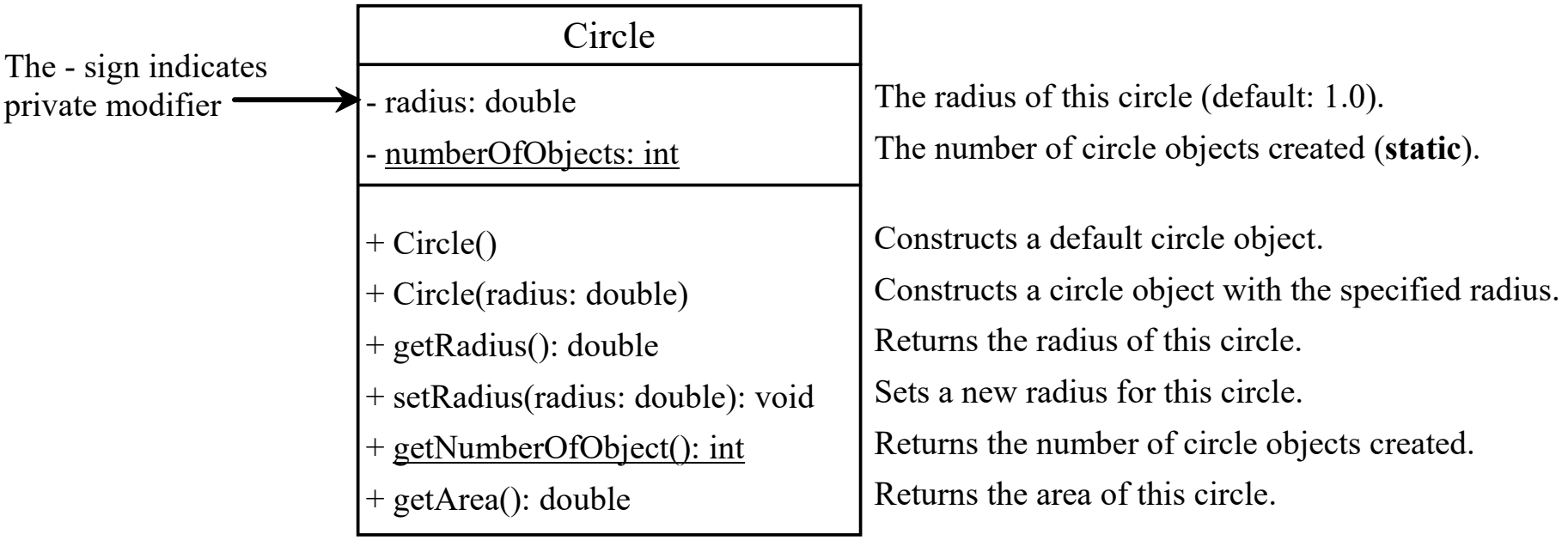
public class C3 {
  cannot access C1;
  can access C2;
}
```

Data Field Encapsulation

Encapsulation is the idea of hiding the class internal details that are not required by clients/users of the class.

Why? To protect data and to make classes easy to maintain and update.

How? Always use private variables!



Visibility Modifiers - Comments - 1

Class members (variables or methods) that are declared with *public visibility* can be referenced/accessed anywhere in the program.

Class members that are declared with *private visibility* can be referenced/accessed only within that class.

Class members declared without a visibility modifier have *default visibility* and can be referenced/accessed by any class in the same package.

Public variables violate encapsulation because they allow class clients to "reach in" and modify the values directly. *Therefore instance variables should not be declared with public visibility.*

Visibility Modifiers - Comments - 2

Methods that provide the object's services must be declared with *public visibility* so that they can be invoked by clients (users of the object).

Public methods are also called *service methods*.

A method created simply to assist a service method is called a *support method.*

Since a support method is not intended to be called by a client, it should be declared with *private visibility.*

Example - 1

```
public class CircleWithPrivateDataFields
{
   private double radius = 1;
   private static int numberOfObjects = 0;

   public CircleWithPrivateDataFields() { numberOfObjects++; }

   public CircleWithPrivateDataFields(double newRadius) {
     radius = newRadius;
     numberOfObjects++;
   }

   public double getRadius() { return radius; }

   public void setRadius(double newRadius) {
     radius = (newRadius >= 0) ? newRadius : 0; //no negative radius
   }

   public static int getNumberOfObjects() {return numberOfObjects; }

   public double getArea() {return radius*radius*Math.PI; }
}
```

```
public class TestCircleWithPrivateDataFields {

  public static void main(String[] args) { // Main method

    // Create a Circle with radius 10.0
    CircleWithPrivateDataFields myCircle =
      new CircleWithPrivateDataFields(10.0);

    System.out.println("The area of the circle of radius "
      + myCircle.getRadius() + " is " + myCircle.getArea());

    // Increase myCircle's radius by 10%
    myCircle.setRadius(myCircle.getRadius() * 1.1);

    System.out.println("The area of the circle of radius "
      + myCircle.getRadius() + " is " + myCircle.getArea());
  }
}
```

Note: variable **radius** cannot be directly accessed. Only through the class methods!

Output:

```
The area of the circle of radius 10.0 is 314.1592653589793
The area of the circle of radius 11.0 is 380.132711084365
```

End of Slides